



UNIVERSITÀ DEGLI STUDI DI TRENTO



CivicTrento

Progetto Ingegneria Del Software

Anno Accademico 2024/2025

Gruppo ID7

Cecilia Cavosi

cecilia.cavosi@studenti.unitn.it
242644

Matilde Prati

matilde.prati@studenti.unitn.it
236042

Elena Rubbo

elena.rubbo@studenti.unitn.it
235858

Università di Trento
Dipartimento di Ingegneria e Scienza dell'Informazione
Via Sommarive 9, 38123
Povo (TN), Italia
E-mail: disi@disi.unitn.it



Indice

1	Sezione Introduttiva	2
2	Sezione Generale	2
2.1	Product Backlog	3
3	Sezione Sprint 2	3
3.1	Sprint Planning	4
3.2	Test Cases	5
3.3	Sprint Review	6
3.4	Sprint Backlog Refinement	6
3.5	Sprint Retrospective	7
4	Sezione Finale	8
4.1	Diagramma del deploy	8
4.2	Stack Tecnologico	9
4.3	Conclusioni	9



1 Sezione Introduttiva

L'applicazione sviluppata propone un innovativo programma di raccolta punti, pensato per premiare i comportamenti virtuosi dei cittadini e incentivare uno stile di vita più responsabile. Attraverso il proprio contributo attivo, gli utenti partecipano alla costruzione di una società più consapevole, sostenibile e orientata al benessere collettivo.

Links:

- link GitHub
- link Apiary
- link apk (condiviso anche da google drive)

Utente di prova:

Accesso:

email: ecmatii@gmail.com

password: Mati123!

Registrazione:

nome: Matilde

cognome: Prati

email: ecmatii@gmail.com

password: Mati123!

codice fiscale: PRTMLD02T63F205C

Numero cartaID: CA00499KT

Dati cittadino:

codice abbonamento: 711217

codice POD: IT001E01001

numero patente: BO5771737J

2 Sezione Generale

Nel progetto è stata adottata una strategia di branching basata sul **Feature Branch Workflow**, che consente di sviluppare ogni nuova funzionalità in un branch separato



per facilitare la gestione del codice e il controllo delle versioni.

2.1 Product Backlog

Product Backlog						
ID	Name	User Story	Importance	Effort	Value	How to Demo
27	Privacy GDPR	Come utente, voglio che i miei dati siano gestiti nel rispetto del GDPR così da garantire la mia privacy.	2,0	2	4	Mostrare che i dati personali sono salvati e trattati secondo policy visibili e approvate.
28	Eliminazione dati	Come utente, voglio che i miei dati siano eliminati dopo 5 anni così da rispettare la normativa GDPR.	2,0	3	6	I dati vengono automaticamente marcati per l'eliminazione dopo 5 anni di inattività.
29	Consenso utenze	Come utente, voglio dover accettare una clausola di autorizzazione per permettere all'app di recuperare i miei dati di consumo dalle utenze, così da essere consapevole e conforme al trattamento dati.	2,0	3	6	Durante la registrazione, viene richiesta esplicitamente l'accettazione dei termini per l'accesso alle utenze.
30	Accesso dati movimento	Come utente dell'app, voglio autorizzare l'accesso ai dati di movimento registrati dalle app del mio dispositivo (es. Google Fit, Salute), così da permettere all'app di monitorare automaticamente i chilometri che percorro ogni giorno.	2,0	8	16	Dopo il login, viene chiesta l'autorizzazione ai dati fitness
31	Sessione singola	Come sistema, devo permettere l'accesso di ciascun utente solo da un dispositivo per volta e bloccare eventuali tentativi di sessione multipla, così da garantire la sicurezza degli account.	1,7	3	5	Tentando l'accesso da un secondo dispositivo, ricevo un messaggio di blocco sessione.
32	Criptazione dati	Come sistema, devo salvare i dati sensibili degli utenti in modo criptato, così da proteggere le informazioni personali e rispettare gli standard di sicurezza.	1,2	5	6	I dati vengono criptati prima di essere salvati.
33	Bonus registrazione	Come utente appena registrato, voglio ricevere automaticamente 800 Civic Coins nel mio portafoglio digitale, in modo da avere subito una dotazione iniziale di moneta civica per poterli utilizzare nella piattaforma.	1,0	1	1	Dopo la registrazione, il mio saldo iniziale mostra 800 Civic Coins.
34	Backup dati	Come sistema, devo creare e gestire dei backup regolari dei dati utenti, così che in caso di guasti o di perdita di dati, si possa ripristinare senza compromettere la continuità del servizio.	1,5	13	19	Simulare una perdita dati e mostrare il ripristino da backup con esito positivo.
35	Scalabilità e performance sistema	Come utente, voglio che il sistema cresca senza rallentamenti e che sia fluido così da non perdere prestazioni nel tempo.	1,5	8	12	Durante test di carico, il sistema mantiene tempi di risposta <1s anche con più utenti connessi. Con l'aumento dei dati, i tempi di caricamento devono restare sotto soglia definita (es. 2s).
36	Referral system	Come utente registrato, voglio avere un link personale o un codice univoco da condividere con altri utenti, così da poter invitarli sulla piattaforma e ricevere punti quando si registrano.	1,0	2	2	Dalla sezione "Impostazioni", viene mostrato un link che, se condiviso, altri possono usare per poter creare a loro volta un account CivicTrento
37	Abbonamenti mezzi pubblici	Come utente, voglio utilizzare i miei Civic Coins per acquistare abbonamenti per i mezzi pubblici, in modo da incentivare una mobilità sostenibile a costo ridotto.	1,0	5	5	Nella sezione "Premi", posso riscattare un abbonamento con Civic Coins.

Link tabella Product Backlog

Ogni funzionalità viene considerata **"Done"** quando il codice è funzionante su tutte le piattaforme previste, sono stati eseguiti i test necessari, la documentazione (incluso il file README) è stata aggiornata, e l'integrazione con il resto del sistema è completata. Per tutti i branch ritenuti completi e stabili, è stato effettuato il merge nel ramo principale.

3 Sezione Sprint 2

L'obiettivo dello Sprint due è la realizzazione e l'implementazione di ulteriori User Story, secondo un ordine di priorità, inoltre ci siamo concentrate sulla correzione degli errori rilevati nello Sprint precedente, al fine di migliorare le diverse funzionalità del sistema.





3.2 Test Cases

Login utente	24	Login con credenziali corrette	<pre> K/U01 - Login con credenziali corrette', steps { 1 => const email = "trento@example.com"; const password = "Password123!"; log[TestData("U01 - Login con credenziali corrette", req); expect(res.body.status).toBe(200); } </pre>	Account loggato@example.com registra con password Password123	Registrazione utente	200 + token	Status code 200	
	25	Login con password errata	<pre> K/U02 - Login con password errata', steps { 1 => const req = await request(app).post("/auth/login").set("email", "trento@example.com", "password", "errata"); log[TestData("U02 - Login con password errata", req); expect(res.status).toBe(400); } </pre>	Account loggato@example.com registra con password diversa da quella valida	Registrazione utente	400	Status code 400	
	26	Login con email non valida	<pre> K/U03 - Login con email non valida', steps { 1 => const req = await request(app).post("/auth/login").set("email", "trentoexample.com", "password", "Password123!"); log[TestData("U03 - Login con email non valida", req); expect(res.status).toBe(400); } </pre>		Registrazione utente	400	Status code 400	
	27	Login con email non valida	<pre> K/U03 - Login con credenziali non registrate', steps { 1 => const req = await request(app).post("/auth/login").set("email", "trento@example.com", "password", "Password123!"); log[TestData("U03 - Login con credenziali non registrate", req); expect(res.status).toBe(400); } </pre>		Registrazione utente	400	Status code 400	
	28	Login con credenziali non registrate	<pre> K/U04 - Registrazione fallita con password semplice', steps { 1 => const req = await request(app).post("/auth/register").set("email", "trento@example.com", "password", "123456", "name", "trento", "cognome", "trento", "CF", "97961234567890123", "cardID", "CARD456789", "gltConsent", true); log[TestData("U04 - Registrazione fallita con password semplice", req); expect(res.status).toBe(400); } </pre>			400	Status code 400	
Password sicura	29	Registrazione fallita con password semplice	<pre> K/U04 - Registrazione fallita con password semplice', steps { 1 => const req = await request(app).post("/auth/register").set("email", "trento@example.com", "password", "123456", "name", "trento", "cognome", "trento", "CF", "97961234567890123", "cardID", "CARD456789", "gltConsent", true); log[TestData("U04 - Registrazione fallita con password semplice", req); expect(res.status).toBe(400); } </pre>			400	Status code 400	
	30	Registrazione fallita con password complessa	<pre> K/U05 - Registrazione fallita con password complessa', steps { 1 => const req = await request(app).post("/auth/register").set("email", "trento@example.com", "password", "Password123!", "name", "trento", "cognome", "trento", "CF", "97961234567890123", "cardID", "CARD456789", "gltConsent", true); log[TestData("U05 - Registrazione fallita con password complessa", req); expect(res.status).toBe(200); } </pre>			200	Status code 200	
Recupero password	31	Recupero password con token valido simulato	<pre> K/U06 - Recupero password con token valido simulato', steps { 1 => const req = await request(app).post("/auth/reset_password").set("token", "valid_token", "email", "trento@example.com"); log[TestData("U06 - Recupero password con token valido simulato", req); expect(res.status).toBe(200); } </pre>	Account registrato con token valido	Registrazione utente	200/400	Status code 200	
	32	Recupero password con token non valido	<pre> K/U07 - Recupero password con token non valido', steps { 1 => const req = await request(app).post("/auth/reset_password").set("token", "invalid_token", "email", "trento@example.com"); log[TestData("U07 - Recupero password con token non valido", req); expect(res.status).toBe(400); } </pre>			400	Status code 400	
	33	Recupero password con token non valido	<pre> K/U07 - Recupero password con token non valido', steps { 1 => const req = await request(app).post("/auth/reset_password").set("token", "invalid_token", "email", "trento@example.com"); log[TestData("U07 - Recupero password con token non valido", req); expect(res.status).toBe(400); } </pre>			400	Status code 400	
Registrazione utente	34	Registrazione con dati esistenti	<pre> K/U08 - Registrazione con dati esistenti', steps { 1 => const req = await request(app).post("/auth/register").set("email", "trento@example.com", "password", "Password123!", "name", "trento", "cognome", "trento", "CF", "97961234567890123", "cardID", "CARD456789", "gltConsent", true); log[TestData("U08 - Registrazione con dati esistenti", req); expect(res.status).toBe(400); } </pre>			400	Status code 400	
	35	Registrazione con dati validi	<pre> K/U09 - Registrazione con dati validi', steps { 1 => const req = await request(app).post("/auth/register").set("email", "trento@example.com", "password", "Password123!", "name", "trento", "cognome", "trento", "CF", "97961234567890123", "cardID", "CARD456789", "gltConsent", true); log[TestData("U09 - Registrazione con dati validi", req); expect(res.status).toBe(200); } </pre>			200/400	Status code 200	
	36	Registrazione con nome vuoto	<pre> K/U10 - Registrazione con nome vuoto', steps { 1 => const req = await request(app).post("/auth/register").set("email", "trento@example.com", "password", "Password123!", "name", "", "cognome", "trento", "CF", "97961234567890123", "cardID", "CARD456789", "gltConsent", true); log[TestData("U10 - Registrazione con nome vuoto", req); expect(res.status).toBe(400); } </pre>			400	Status code 400	
	37	Registrazione con cognome vuoto	<pre> K/U11 - Registrazione con cognome vuoto', steps { 1 => const req = await request(app).post("/auth/register").set("email", "trento@example.com", "password", "Password123!", "name", "trento", "cognome", "", "CF", "97961234567890123", "cardID", "CARD456789", "gltConsent", true); log[TestData("U11 - Registrazione con cognome vuoto", req); expect(res.status).toBe(400); } </pre>			400	Status code 400	
	38	Registrazione con email vuota	<pre> K/U12 - Registrazione con email vuota', steps { 1 => const req = await request(app).post("/auth/register").set("email", "", "password", "Password123!", "name", "trento", "cognome", "trento", "CF", "97961234567890123", "cardID", "CARD456789", "gltConsent", true); log[TestData("U12 - Registrazione con email vuota", req); expect(res.status).toBe(400); } </pre>			400	Status code 400	
	39	Registrazione con password vuota	<pre> K/U13 - Registrazione con password vuota', steps { 1 => const req = await request(app).post("/auth/register").set("email", "trento@example.com", "password", "", "name", "trento", "cognome", "trento", "CF", "97961234567890123", "cardID", "CARD456789", "gltConsent", true); log[TestData("U13 - Registrazione con password vuota", req); expect(res.status).toBe(400); } </pre>			400	Status code 400	
	40	Registrazione con codice fiscale vuoto	<pre> K/U14 - Registrazione con codice fiscale vuoto', steps { 1 => const req = await request(app).post("/auth/register").set("email", "trento@example.com", "password", "Password123!", "name", "trento", "cognome", "trento", "CF", "", "cardID", "CARD456789", "gltConsent", true); log[TestData("U14 - Registrazione con codice fiscale vuoto", req); expect(res.status).toBe(400); } </pre>			400	Status code 400	
	41	Registrazione con carta identità vuota	<pre> K/U15 - Registrazione con carta identità vuota', steps { 1 => const req = await request(app).post("/auth/register").set("email", "trento@example.com", "password", "Password123!", "name", "trento", "cognome", "trento", "CF", "97961234567890123", "cardID", "", "gltConsent", true); log[TestData("U15 - Registrazione con carta identità vuota", req); expect(res.status).toBe(400); } </pre>			400	Status code 400	
	42	Registrazione con email non valida	<pre> K/U16 - Registrazione con email non valida', steps { 1 => const req = await request(app).post("/auth/register").set("email", "trentoexample.com", "password", "Password123!", "name", "trento", "cognome", "trento", "CF", "97961234567890123", "cardID", "CARD456789", "gltConsent", true); log[TestData("U16 - Registrazione con email non valida", req); expect(res.status).toBe(400); } </pre>			400	Status code 400	
	43	Registrazione con email già esistente	<pre> K/U17 - Registrazione con email già esistente', steps { 1 => const req = await request(app).post("/auth/register").set("email", "trento@example.com", "password", "Password123!", "name", "trento", "cognome", "trento", "CF", "97961234567890123", "cardID", "CARD456789", "gltConsent", true); log[TestData("U17 - Registrazione con email già esistente", req); expect(res.status).toBe(400); } </pre>			400	Status code 400	
	44	Registrazione con carta fiscale non valida	<pre> K/U18 - Registrazione con carta fiscale non valida', steps { 1 => const req = await request(app).post("/auth/register").set("email", "trento@example.com", "password", "Password123!", "name", "trento", "cognome", "trento", "CF", "", "cardID", "CARD456789", "gltConsent", true); log[TestData("U18 - Registrazione con carta fiscale non valida", req); expect(res.status).toBe(400); } </pre>			400	Status code 400	
	45	Registrazione con carta identità non valida	<pre> K/U19 - Registrazione con carta identità non valida', steps { 1 => const req = await request(app).post("/auth/register").set("email", "trento@example.com", "password", "Password123!", "name", "trento", "cognome", "trento", "CF", "97961234567890123", "cardID", "", "gltConsent", true); log[TestData("U19 - Registrazione con carta identità non valida", req); expect(res.status).toBe(400); } </pre>			400	Status code 400	
Conservazione GDPR	46	Registrazione fallita senza consenso GDPR	<pre> K/U20 - Registrazione fallita senza consenso GDPR', steps { 1 => const req = await request(app).post("/auth/register").set("email", "trento@example.com", "password", "Password123!", "name", "trento", "cognome", "trento", "CF", "97961234567890123", "cardID", "CARD456789", "gltConsent", false); log[TestData("U20 - Registrazione fallita senza consenso GDPR", req); expect(res.status).toBe(400); } </pre>			400	Status code 400	
	47	Conservazione account riuscita	<pre> K/U21 - Conservazione account riuscita', steps { 1 => const req = await request(app).post("/auth/consent").set("email", "trento@example.com", "password", "Password123!"); log[TestData("U21 - Conservazione account riuscita", req); expect(res.status).toBe(200); } </pre>	Utente accettato	Login utente	200/400	Status code 200	
Modifica password	48	Modifica password con valore valido	<pre> K/U22 - Modifica password con valore valido', steps { 1 => const req = await request(app).post("/auth/reset_password").set("token", "valid_token", "email", "trento@example.com", "password", "Password123!"); log[TestData("U22 - Modifica password con valore valido", req); expect(res.status).toBe(200); } </pre>	Account con password già valida nel database	Registrazione utente a login	200	Status code 200	
	49	Modifica password con valore valido	<pre> K/U22 - Modifica password con valore valido', steps { 1 => const req = await request(app).post("/auth/reset_password").set("token", "valid_token", "email", "trento@example.com", "password", "Password123!"); log[TestData("U22 - Modifica password con valore valido", req); expect(res.status).toBe(200); } </pre>			200	Status code 200	

[illegible]

[Link tabella Test Cases](#)

3.3 Sprint Review

Durante l'implementazione delle nuove funzionalità, abbiamo riscontrato alcuni problemi che non si erano manifestati nelle fasi iniziali della demo. Proseguendo con i test e l'integrazione, sono emersi nuovi errori legati al comportamento del sistema in condizioni reali o non previste nei casi base. Nel corso dello sprint, ci siamo concentrate sull'analisi e risoluzione di questi imprevisti, lavorando in modo iterativo per correggere i bug rilevati e garantire una maggiore stabilità del sistema. Questo processo ci ha permesso di migliorare sia la qualità del codice, sia l'affidabilità complessiva dell'applicazione, adattandoci rapidamente ai problemi emersi grazie al confronto continuo all'interno del team.

3.4 Sprint Backlog Refinement

Durante l'avanzamento dello sviluppo dell'applicazione, abbiamo rivalutato le User Story relative alla figura dell'amministratore e riteniamo che non siano più coerenti con gli obiettivi del progetto. L'app è infatti concepita per premiare i cittadini che adottano comportamenti virtuosi, come evitare sanzioni, consumare responsabilmente le utenze, utilizzare i trasporti pubblici o partecipare al voto.

In questo contesto, la figura dell'amministratore — intesa inizialmente come un dipendente comunale con pieni poteri decisionali sull'assegnazione dei punti — risulta poco adatta e in contrasto con la visione di un sistema premiante automatico, trasparente e imparziale. Riteniamo quindi che, se mantenuta, la figura dell'amministratore debba avere un ruolo più tecnico e limitato, volto esclusivamente alla gestione e alla risoluzione di eventuali malfunzionamenti del sistema, senza intervenire direttamente sull'assegnazione dei punti. In ogni caso, la realizzazione di tale componente richiederebbe uno sprint dedicato, qualora in futuro se ne ritenesse ancora necessaria l'introduzione.



In compenso, abbiamo anticipato l'implementazione di alcune user story successive, più lineari, e ci siamo concentrati sulla rifinitura delle funzionalità già sviluppate nel primo sprint, a cui avevamo dedicato meno tempo e attenzione. Questo ci ha permesso di correggere errori emersi durante i test e migliorare la qualità delle componenti esistenti.

3.5 Sprint Retrospective

In questo secondo sprint, siamo riuscite ad applicare il metodo Agile in modo più consapevole ed efficace rispetto al primo. La suddivisione delle attività è avvenuta con maggiore attenzione alle competenze e al carico di lavoro individuale, favorendo un buon bilanciamento tra autonomia e collaborazione.

Il confronto all'interno del team è stato costante e costruttivo: abbiamo discusso diverse soluzioni per affrontare problemi emersi durante l'implementazione e ci siamo supportate nel superare gli ostacoli tecnici.

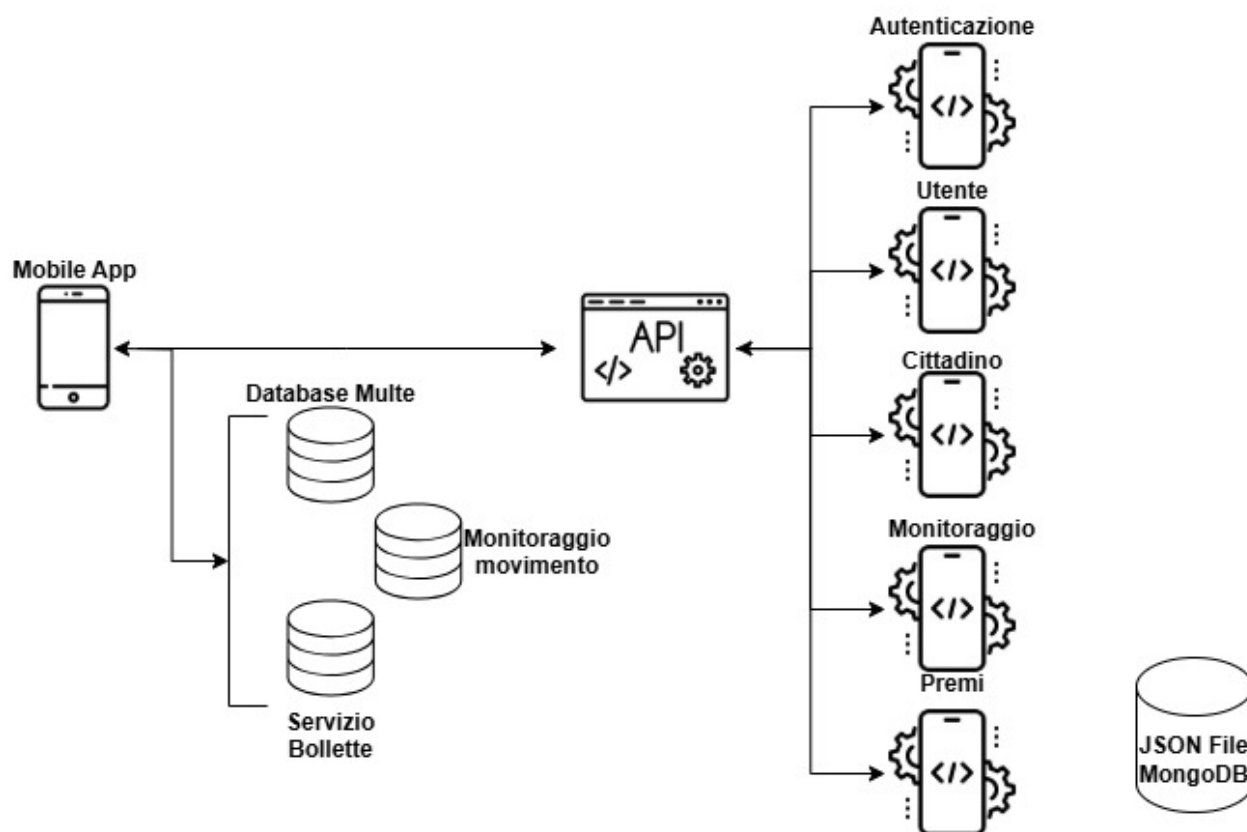
Una criticità riscontrata ha riguardato l'utilizzo della repository, dove si sono verificati alcuni problemi di sincronizzazione e conflitti tra le modifiche, che hanno richiesto tempo aggiuntivo per essere risolti. Questo aspetto ha messo in evidenza la necessità di migliorare l'organizzazione del lavoro condiviso tramite Git e rafforzare le buone pratiche di versionamento.

Un'ulteriore criticità ha riguardato l'implementazione della User Story relativa al monitoraggio dei movimenti a piedi. Lato IOS Non siamo riuscite a far sì che l'applicazione prelevasse i dati relativi ai chilometri percorsi tramite l'app di monitoraggio già integrata sul dispositivo, lato Android non siamo riuscite a farla funzionare, a causa di problemi legati alla compatibilità tra diverse versioni.



4 Sezione Finale

4.1 Diagramma del deploy





4.2 Stack Tecnologico

Componente	Tecnologia / Linguaggio	Descrizione / Ruolo
Frontend	Flutter	Framework per la creazione dell'interfaccia utente dell'app mobile
Linguaggio	Dart	Linguaggio principale per lo sviluppo delle schermate e della logica client-side
Backend	Node.js	Gestione delle API REST e della logica lato server
Linguaggio	JavaScript	Linguaggio utilizzato per lo sviluppo del backend in ambiente Node.js
Database	File JSON (temporaneo) + MongoDB (permanente)	Archiviazione locale per test e database NoSQL per la persistenza dati
Autenticazione	Gestita su backend	Login, registrazione e verifica credenziali
Controllo versioni	Git + GitHub	Gestione delle versioni del codice e collaborazione
Documentazione	Wiki(GitHub)	Documentazione completa e consultabile della applicazione
API REST	Express + documentazione Apiary	Endpoints definiti con Express e documentati per gli sviluppatori tramite Apiary
Deployment API	Locale	Hosting delle API (es. Render, Railway, ecc.)
Documentazione API	Apiary	Documentazione completa e consultabile delle API
Test API	Jest + Supertest	Test automatici delle rotte backend con verifica di status code e risposte
Testing UI/API	Swagger UI	Verifica del funzionamento e struttura delle API REST

Link Stack Tecnologico

4.3 Conclusioni

Siamo riuscite a raggiungere gli obiettivi: incentivare comportamenti virtuosi tramite CivicCoins con monitoraggio dati, storico e sistema premi.

I limiti attuali sono la mancata integrazione con enti esterni e funzionalità avanzate. I futuri sviluppi prevedono miglior integrazione istituzionale, dashboard, gamification e notifiche.